

```
$this->registerPolicies();
Gate::define('update-post', function ($user, $post) {
    return $user->id == $post->user_id;
});}
```

Multiple gates can be defined at the same time using another method called 'resource',

```
Gate::resource('posts', 'PostPolicy');
```

The various abilities such as view, create, delete etc. are pre defined. This can be bypassed by the 'resource' method but it would include manually created logic for the gates. For authorisation action using gates, the 'allows' or 'denies' methodology is used.

```
if (Gate::allows('update-post', $post)) {
    // The current user can update the post...
}
if (Gate::denies('update-post', $post)) {
    // The current user can't update the post...
}
```

Policies

Just like controllers, Policies are classes that organise the authorisation logic around a particular model which may be pre defined or user defined. The 'make:policy' command is used to generate an empty policy class which holds the logic for the functionality of that particular policy. The generated policy would be stored in the 'app/Policies' directory of the project. After creating a policy using this simple method, it needs to be registered in order to be recognised. The 'AuthServiceProvider' contains a 'policies' property for mapping the eloquent database models with the various policies.

```
<?php
namespace App\Providers;
use App\Post;
use App\Policies\PostPolicy;
use Illuminate\Support\Facades\Gate;
use Illuminate\Foundation\Support\Providers\AuthService-
Provider as ServiceProvider;
class AuthServiceProvider extends ServiceProvider
{
    /**
```